

Lista de Exercícios – Detecção e Prevenção de Impasses em C

Prof. Vinícius S. Borges

Instruções

Em cada questão, analise cuidadosamente o código em C e responda aos itens abaixo:

1. Diga se há **impasse**, **possibilidade de impasse** ou apenas **espera temporária**.
2. Identifique e justifique a presença ou ausência das **quatro condições necessárias para impasse**:
 - exclusão mútua;
 - posse e espera;
 - não-preempção;
 - espera circular.
3. Caso exista impasse ou possibilidade de impasse, explique **como impedir o problema**.
4. Escreva uma **versão corrigida em C** que elimine o risco identificado.

Questão 1 – Transferência bancária com ordem invertida

```
1 #include <pthread.h>
2 #include <stdio.h>
3 #include <unistd.h>
4
5 pthread_mutex_t conta1 = PTHREAD_MUTEX_INITIALIZER;
6 pthread_mutex_t conta2 = PTHREAD_MUTEX_INITIALIZER;
7
8 void *transferencia_1(void *arg) {
9     pthread_mutex_lock(&conta1);
10    sleep(1);
11    pthread_mutex_lock(&conta2);
12
13    printf("Transferencia 1 concluida\n");
```

```

14
15     pthread_mutex_unlock(&conta2);
16     pthread_mutex_unlock(&conta1);
17     return NULL;
18 }
19
20 void *transferencia_2(void *arg) {
21     pthread_mutex_lock(&conta2);
22     sleep(1);
23     pthread_mutex_lock(&conta1);
24
25     printf("Transferencia 2 concluida\n");
26
27     pthread_mutex_unlock(&conta1);
28     pthread_mutex_unlock(&conta2);
29     return NULL;
30 }

```

Responda:

- a) Há impasse ou possibilidade de impasse?
- b) Quais das quatro condições necessárias estão presentes?
- c) Como impedir esse problema?
- d) Reescreva o código em C de forma segura.

Questão 2 – Três threads e ciclo completo

```

1  #include <pthread.h>
2  #include <stdio.h>
3  #include <unistd.h>
4
5  pthread_mutex_t r1 = PTHREAD_MUTEX_INITIALIZER;
6  pthread_mutex_t r2 = PTHREAD_MUTEX_INITIALIZER;
7  pthread_mutex_t r3 = PTHREAD_MUTEX_INITIALIZER;
8
9  void *t1(void *arg) {
10     pthread_mutex_lock(&r1);
11     sleep(1);
12     pthread_mutex_lock(&r2);
13     printf("t1 executou\n");
14     pthread_mutex_unlock(&r2);
15     pthread_mutex_unlock(&r1);
16     return NULL;
17 }
18
19 void *t2(void *arg) {
20     pthread_mutex_lock(&r2);
21     sleep(1);

```

```

22     pthread_mutex_lock(&r3);
23     printf("t2 executou\n");
24     pthread_mutex_unlock(&r3);
25     pthread_mutex_unlock(&r2);
26     return NULL;
27 }
28
29 void *t3(void *arg) {
30     pthread_mutex_lock(&r3);
31     sleep(1);
32     pthread_mutex_lock(&r1);
33     printf("t3 executou\n");
34     pthread_mutex_unlock(&r1);
35     pthread_mutex_unlock(&r3);
36     return NULL;
37 }

```

Responda:

- Existe espera circular? Explique.
- As quatro condições de impasse estão presentes?
- Como prevenir o problema?
- Reescreva o código usando uma política segura de aquisição de recursos.

Questão 3 – Função auxiliar que esconde o risco

```

1  #include <pthread.h>
2  #include <stdio.h>
3  #include <unistd.h>
4
5  pthread_mutex_t log_mutex = PTHREAD_MUTEX_INITIALIZER;
6  pthread_mutex_t banco_mutex = PTHREAD_MUTEX_INITIALIZER;
7
8  void registrar_log() {
9      pthread_mutex_lock(&log_mutex);
10     usleep(1000);
11     pthread_mutex_unlock(&log_mutex);
12 }
13
14 void atualizar_banco() {
15     pthread_mutex_lock(&banco_mutex);
16     usleep(1000);
17     registrar_log();
18     pthread_mutex_unlock(&banco_mutex);
19 }
20
21 void *thread1(void *arg) {
22     pthread_mutex_lock(&log_mutex);

```

```

23     usleep(1000);
24     atualizar_banco();
25     pthread_mutex_unlock(&log_mutex);
26     return NULL;
27 }

```

Responda:

- Por que esse código é mais difícil de analisar do que os anteriores?
- Pode ocorrer impasse? Justifique.
- Proponha uma segunda thread que complete o cenário de deadlock.
- Reescreva a solução em C de modo a evitar o problema.

Questão 4 – trylock e risco de livelock

```

1  #include <pthread.h>
2  #include <stdio.h>
3  #include <unistd.h>
4
5  pthread_mutex_t m1 = PTHREAD_MUTEX_INITIALIZER;
6  pthread_mutex_t m2 = PTHREAD_MUTEX_INITIALIZER;
7
8  void *t1(void *arg) {
9      while (1) {
10         pthread_mutex_lock(&m1);
11         if (pthread_mutex_trylock(&m2) == 0) {
12             break;
13         }
14         pthread_mutex_unlock(&m1);
15         usleep(100);
16     }
17
18     printf("t1 executando\n");
19     pthread_mutex_unlock(&m2);
20     pthread_mutex_unlock(&m1);
21     return NULL;
22 }
23
24 void *t2(void *arg) {
25     while (1) {
26         pthread_mutex_lock(&m2);
27         if (pthread_mutex_trylock(&m1) == 0) {
28             break;
29         }
30         pthread_mutex_unlock(&m2);
31         usleep(100);
32     }
33 }

```

```

34     printf("t2 executando\n");
35     pthread_mutex_unlock(&m1);
36     pthread_mutex_unlock(&m2);
37     return NULL;
38 }

```

Responda:

- a) Há deadlock clássico? Explique.
- b) Há outro problema concorrente possível? Qual?
- c) Quais condições de impasse foram quebradas e quais permanecem?
- d) Reescreva o código de maneira mais robusta.

Questão 5 – Recursos adquiridos em função recursiva

```

1  #include <pthread.h>
2  #include <stdio.h>
3  #include <unistd.h>
4
5  pthread_mutex_t A = PTHREAD_MUTEX_INITIALIZER;
6  pthread_mutex_t B = PTHREAD_MUTEX_INITIALIZER;
7
8  void processar(int nivel) {
9      if (nivel == 0) return;
10
11     pthread_mutex_lock(&A);
12     usleep(1000);
13
14     if (nivel % 2 == 0) {
15         pthread_mutex_lock(&B);
16         pthread_mutex_unlock(&B);
17     }
18
19     processar(nivel - 1);
20     pthread_mutex_unlock(&A);
21 }
22
23 void *thread1(void *arg) {
24     processar(2);
25     return NULL;
26 }
27
28 void *thread2(void *arg) {
29     pthread_mutex_lock(&B);
30     usleep(1000);
31     pthread_mutex_lock(&A);
32
33     pthread_mutex_unlock(&A);

```

```

34     pthread_mutex_unlock(&B);
35     return NULL;
36 }

```

Responda:

- Há auto-bloqueio, impasse entre threads, ambos ou nenhum?
- Como a recursão interfere na análise?
- Quais condições de impasse aparecem no cenário?
- Reescreva o código de forma segura.

Questão 6 – Produtor, manutenção e função auxiliar

```

1  #include <pthread.h>
2  #include <stdio.h>
3  #include <unistd.h>
4
5  pthread_mutex_t fila_mutex = PTHREAD_MUTEX_INITIALIZER;
6  pthread_mutex_t estat_mutex = PTHREAD_MUTEX_INITIALIZER;
7
8  void atualizar_estatisticas() {
9      pthread_mutex_lock(&estat_mutex);
10     usleep(1000);
11     pthread_mutex_unlock(&estat_mutex);
12 }
13
14 void *produtor(void *arg) {
15     pthread_mutex_lock(&fila_mutex);
16     usleep(1000);
17     atualizar_estatisticas();
18     pthread_mutex_unlock(&fila_mutex);
19     return NULL;
20 }
21
22 void *manutencao(void *arg) {
23     pthread_mutex_lock(&estat_mutex);
24     usleep(1000);
25     pthread_mutex_lock(&fila_mutex);
26
27     pthread_mutex_unlock(&fila_mutex);
28     pthread_mutex_unlock(&estat_mutex);
29     return NULL;
30 }

```

Responda:

- O deadlock depende do escalonamento? Explique.
- A função `atualizar_estatisticas()` contribui para posse e espera? Justifique.

- c) Apresente duas formas de prevenção.
- d) Reescreva o código em C corrigindo o problema.

Questão 7 – Impressora e spooler com modelagem incorreta

```
1 #include <pthread.h>
2 #include <stdio.h>
3 #include <unistd.h>
4
5 pthread_mutex_t impressora = PTHREAD_MUTEX_INITIALIZER;
6 pthread_mutex_t spooler = PTHREAD_MUTEX_INITIALIZER;
7
8 void *usuario1(void *arg) {
9     pthread_mutex_lock(&impressora);
10    usleep(1000);
11    pthread_mutex_lock(&spooler);
12
13    printf("usuario1 imprimindo\n");
14
15    pthread_mutex_unlock(&spooler);
16    pthread_mutex_unlock(&impressora);
17    return NULL;
18 }
19
20 void *usuario2(void *arg) {
21    pthread_mutex_lock(&spooler);
22    usleep(1000);
23    pthread_mutex_lock(&impressora);
24
25    printf("usuario2 imprimindo\n");
26
27    pthread_mutex_unlock(&impressora);
28    pthread_mutex_unlock(&spooler);
29    return NULL;
30 }
```

Responda:

- a) Há impasse ou possibilidade de impasse?
- b) Por que essa modelagem contradiz a ideia correta de *spooling*?
- c) Quais condições de impasse aparecem?
- d) Reescreva a lógica de modo que apenas o spooler use a impressora diretamente.

Questão 8 – Deadlock oculto por abstração em módulos

```

1 #include <pthread.h>
2 #include <stdio.h>
3 #include <unistd.h>
4
5 pthread_mutex_t cache_mutex = PTHREAD_MUTEX_INITIALIZER;
6 pthread_mutex_t disco_mutex = PTHREAD_MUTEX_INITIALIZER;
7
8 void gravar_disco() {
9     pthread_mutex_lock(&disco_mutex);
10    usleep(1000);
11    pthread_mutex_unlock(&disco_mutex);
12 }
13
14 void atualizar_cache() {
15     pthread_mutex_lock(&cache_mutex);
16     usleep(1000);
17     gravar_disco();
18     pthread_mutex_unlock(&cache_mutex);
19 }
20
21 void flush_disco_para_cache() {
22     pthread_mutex_lock(&disco_mutex);
23     usleep(1000);
24     pthread_mutex_lock(&cache_mutex);
25
26     pthread_mutex_unlock(&cache_mutex);
27     pthread_mutex_unlock(&disco_mutex);
28 }

```

Responda:

- Por que a divisão em funções dificulta enxergar o impasse?
- Descreva um cenário com duas threads que gere deadlock.
- Identifique as quatro condições de impasse.
- Corrija a arquitetura do acesso aos recursos.

Questão 9 – Timeout parcial e solução inconsistente

```

1 #include <pthread.h>
2 #include <stdio.h>
3 #include <unistd.h>
4
5 pthread_mutex_t x = PTHREAD_MUTEX_INITIALIZER;
6 pthread_mutex_t y = PTHREAD_MUTEX_INITIALIZER;
7
8 void *t1(void *arg) {
9     pthread_mutex_lock(&x);
10    sleep(1);

```



```

11
12     if (pthread_mutex_trylock(&y) != 0) {
13         pthread_mutex_unlock(&x);
14         return NULL;
15     }
16
17     printf("t1 concluiu\n");
18     pthread_mutex_unlock(&y);
19     pthread_mutex_unlock(&x);
20     return NULL;
21 }
22
23 void *t2(void *arg) {
24     pthread_mutex_lock(&y);
25     sleep(1);
26     pthread_mutex_lock(&x);
27
28     printf("t2 concluiu\n");
29     pthread_mutex_unlock(&x);
30     pthread_mutex_unlock(&y);
31     return NULL;
32 }

```

Responda:

- a) Ainda pode ocorrer impasse ou outro problema de concorrência?
- b) O timeout parcial resolve o sistema inteiro? Justifique.
- c) Quais condições foram quebradas para t1 e quais continuam válidas para t2?
- d) Reescreva uma solução consistente.

Questão 10 – Múltiplos recursos com ciclo de dependência

```

1  #include <pthread.h>
2  #include <stdio.h>
3  #include <unistd.h>
4
5  pthread_mutex_t a = PTHREAD_MUTEX_INITIALIZER;
6  pthread_mutex_t b = PTHREAD_MUTEX_INITIALIZER;
7  pthread_mutex_t c = PTHREAD_MUTEX_INITIALIZER;
8
9  void *t1(void *arg) {
10     pthread_mutex_lock(&a);
11     usleep(1000);
12     pthread_mutex_lock(&b);
13     usleep(1000);
14     pthread_mutex_lock(&c);
15

```

```

16     printf("t1 executou\n");
17
18     pthread_mutex_unlock(&c);
19     pthread_mutex_unlock(&b);
20     pthread_mutex_unlock(&a);
21     return NULL;
22 }
23
24 void *t2(void *arg) {
25     pthread_mutex_lock(&c);
26     usleep(1000);
27     pthread_mutex_lock(&b);
28
29     printf("t2 executou\n");
30
31     pthread_mutex_unlock(&b);
32     pthread_mutex_unlock(&c);
33     return NULL;
34 }
35
36 void *t3(void *arg) {
37     pthread_mutex_lock(&b);
38     usleep(1000);
39     pthread_mutex_lock(&a);
40
41     printf("t3 executou\n");
42
43     pthread_mutex_unlock(&a);
44     pthread_mutex_unlock(&b);
45     return NULL;
46 }

```

Responda:

- Existe ciclo de espera entre as três threads? Explique.
- Diferencie dependência parcial e circularidade completa neste exemplo.
- Identifique rigorosamente as quatro condições de impasse.
- Reescreva o programa usando uma ordem global de aquisição.